TEMCO LEARN

T3000 API & Protocol

Database Management API · built Nov 13, 2026

Database Management API

Overview

The Database Management API provides application-level settings storage, database partitioning, maintenance, and monitoring.

Common Operations

Get Settings

TYPESCRIPT

```typescript
const settings = await fetch('http://localhost:9103/api/db_management/settings')
  .then(res => res.json());
```

Save Setting

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/db_management/settings', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    key: 'theme',
    value: 'dark'
  })
});
```

Get Database Stats

```typescript
const stats = await fetch('http://localhost:9103/api/database/stats')
  .then(res => res.json());

console.log(stats);
// {
//   size: 52428800,
//   tables: 45,
//   total_records: 152340
// }
```

Settings Endpoints

GET /api/db_management/settings

Description: Get all application settings

Request:

```http
GET /api/db_management/settings HTTP/1.1
```

Response:

```json
[
  {
    "id": 1,
    "key": "theme",
    "value": "dark",
    "created_at": "2026-01-01T10:00:00Z",
    "updated_at": "2026-01-09T08:00:00Z"
  }
]
```

Used By: SettingsPage.tsx

POST /api/db_management/settings

Description: Create or update setting

Request:

```HTTP
POST /api/db_management/settings HTTP/1.1
Content-Type: application/json

{
  "key": "auto_refresh",
  "value": "true"
}
```

Response:

```JSON
{
  "id": 2,
  "key": "auto_refresh",
  "value": "true"
}
```

DELETE /api/db_management/settings/:key

Description: Delete setting by key

Parameters:

- `key` (path, string) - Setting key

Request:

```HTTP
DELETE /api/db_management/settings/theme HTTP/1.1
```

Response:

```
HTTP/1.1 204 No Content
```

Database Maintenance

GET /api/database/stats

Description: Get database statistics

Request:

```
GET /api/database/stats HTTP/1.1
```

Response:

```
{
  "size": 52428800,
  "tables": 45,
  "total_records": 152340,
  "indexes": 62,
  "fragmentation": 5.2
}
```

Used By: DatabaseStats.tsx

POST /api/db_management/tools/vacuum

Description: Vacuum database to reclaim space

Request:

```
POST /api/db_management/tools/vacuum HTTP/1.1
```

Response:

```JSON
{
  "success": true,
  "size_before": 52428800,
  "size_after": 41943040,
  "space_reclaimed": 10485760
}
```

Used By: DatabaseMaintenance.tsx

POST /api/database/cleanup/old

Description: Delete old database files

Request:

```HTTP
POST /api/database/cleanup/old HTTP/1.1
Content-Type: application/json

{
  "days": 30
}
```

Response:

```JSON
{
  "success": true,
  "files_deleted": 12
}
```

GET /api/db_management/partitions

Description: Get partition information

Request:

```HTTP
GET /api/db_management/partitions HTTP/1.1
```

Response:

```JSON
[
  {
    "name": "trendlog_2026_01",
    "table": "TRENDLOG_DATA",
    "start_date": "2026-01-01",
    "end_date": "2026-01-31",
    "records": 124800
  }
]
```

Database Schema

```SQL
CREATE TABLE APP_SETTINGS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    key TEXT UNIQUE NOT NULL,
    value TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

TypeScript Types

TYPESCRIPT

```typescript
export interface AppSetting {
  id: number;
  key: string;
  value: string;
  created_at: string;
  updated_at: string;
}

export interface DatabaseStats {
  size: number;
  tables: number;
  total_records: number;
  indexes: number;
  fragmentation: number;
}
```